Employee Performance & Salary Analyst

```
import csv

import random

from datetime import datetime,timedelta

import pandas as pd

import numpy as np


file_name=("employee_data.csv")


employee_id=[101,102,103,104,105,106,107]

employee_names=["Amit","Rahul","Sneha","Pooja","Vikas","Neha","Rohit","Anajali","Sagar","Priya","Akash
","Kiran","Meena","Rakesh","Komal","Nitin","Sachin","Swati","Deepak","Riya"]

departments=["IT","HR","Sales","Finance","Marketing"]

job_roles=["Data Analyst","Developer","HR Executive","Data Science","Data Cleaning","Operator"]

age=[17,55,45,28,25,36,63]

genders=["Male","Female"]

experience_years=[5,1,2,3,4,5,8]

monthly_salary=[25000,23000,45000,26000,55000,56000,60000]

performance_score=[2,5,5,6,3,4,8]

projects_completed=[4,5,6,8,1,2,3]

training_hours=[40,20,45,20,12,30,50]

joining_date=['2026-01-01','2026-01-03','2026-01-05','2026-01-09','2026-01-12','2026-01-19','2026-01-
22']

attritions=["Yes","No"]


#CSV Header


header=["Employee ID","Employee Name","Department","Job Role","Age","Gender","Experience
Years","Monthly Salary","Performance Score","Project Completed","Training Hours","Joining
Date","Attrition"]
```

```
#Create CSV File


with open(file_name,"w",newline="")as file:
   writer=csv.writer(file)
   writer.writerow(header)
   start_date=datetime(2026,1,1)
   for i in range(1,1000):
      employee_id=f"C{i:03}"
      employee_name=random.choice(employee_names)
      department=random.choice(departments)
      job_role=random.choice(job_roles)
      age=random.randint(21,65)
      gender=random.choice(genders)
      experience_years=random.randint(1,12)
      monthly_salary=random.randrange(20000,55000,1000)
      performance_score=random.randint(1,10)
      projects_completed=random.randint(1,15)
      training_hours=random.randint(10,50)
      joining_date=(start_date+timedelta(days=random.randint(0,364)))
      attrition=random.choice(attritions)



writer.writerow([employee_id,employee_name,department,job_role,age,gender,experience_years,monthly_salary,performance_score,projects_completed,training_hours,joining_date.strftime("%d-%m-%Y"),attrition])

print("================================================================================")

print("employee_data.csv Created Successfully!")

print("Total Records: 1000")

print("================================================================================")
```

```
print("============================Practical Task List============================")
df=pd.read_csv("employee_data.csv")
print(df)
print("======First 10 and Last 5 Records=======")
print("First 10:",df.head(10))
print("Last 5:",df.tail(5))
print("======Find The Number Of Rows & Columns======")
print("Number Of Rows & Columns:",df.shape)
print("======Column Names And Data Types======")
print("Columns Names:",df.columns)
print("Data Types:",df.dtypes)
print("======Statistical Summary Describe======")
print("Describe:",df.describe())

print("============================Data Cleaning============================")
print("======Remove Duplicate Records======")
print("Remove Duplicate:",df.duplicated().sum())
print("======Missing Values In Each Column======")
print("Missing Values:",df.isnull().sum())
print("======Missing Salary Values With The Departments Median Salary======")
m=df['Monthly Salary']=df['Monthly Salary'].fillna(df.groupby('Department')['Monthly
Salary'].transform('median'))
print("Missing Salary Values:",m)
print("======Inconsistent Department Names And Extra Space======")
df['Department']=df['Department'].str.strip()
print(df['Department'].unique())
print("======Joining Date Into Datetime Formate======")
date=df['Joining Date']=pd.to_datetime(df['Joining Date'],format='%d-%-m-%Y')
print(date)
print("======Negative Salaries And Invalid Performance Scores======")
print(df[['Monthly Salary','Performance Score']].isnull().sum())
```

```
ne=df[df['Monthly Salary']<0|(df['Performance Score']>100)]

print(ne)


print("============================Data
Transformation==============================")

print("======Create Annual Salary======")

df['Annual Salary']=df['Monthly Salary']*12

print(df[['Employee Name','Monthly Salary','Annual Salary']])

print("======Experience Level======")

df['Experience Level']=np.where(df['Experience Years']<2,'Fresher',np.where(df['Experience
Years']<7,'Mid-Level','Senior'))

print(df)

print("======Performance Category======")

df['Performance Category']=np.where(df['Performance Score']<2,'Low',np.where(df['Performance
Score']<7,'Average','High'))

print(df)

print("======Salary Band=======")

df['Salary Band']=np.where(df['Monthly Salary']<10000,'Low',np.where(df['Monthly
Salary']<25000,'Medium','High'))

print(df)

print("======Years In Company From The Joining Date=======")

df['Joining Date']=pd.to_datetime(df['Joining Date'])

df['year']=df['Joining Date'].dt.year

print(df[['year','Joining Date']])


print("============================Business
Analysis==============================")

print("=======Average Salary By Department=======")

avg_salary=df.groupby('Department')['Monthly Salary'].mean().sort_values(ascending=False)

print(avg_salary)

print("=======Highest Paid Employee=======")

highest=df.groupby('Employee Name')['Monthly Salary'].sum().nlargest(5)
```

```
print(highest)
print("=======Top 10 Employees By Performance Score=======")
top10=df.groupby('Employee Name')['Performance Score'].sum().nlargest(10)
print(top10)
print("=======Average Salary By Job Role=======")
avg_sal=df.groupby('Job Role')['Monthly Salary'].mean().sort_values(ascending=False)
print(avg_sal)
print("=======Average Experience Across Department=======")
avg_exp=df.groupby('Department')['Experience Years'].mean().sort_values(ascending=False)
print(avg_exp)
print("=======Employee Attrition Percentage=======")
total_emp=len(df)
emp=df[df['Attrition']=='Yes'].shape[0]
prc=(emp/total_emp)*100
print("Total Employee:",total_emp)
print("Employee Left:",emp)
print(f"Attrition Percentage:{prc:.2f}%")

print("=======Employee With High Salaries But Low Performance=======")
print(df[df['Salary Band']=='High'],df[df['Performance Category']=='Low'])

print("==============================Advanced
Pandas==============================")
print("=======Summarize Department Performance=======")
summarize=df.groupby('Department')['Performance Score'].agg(["count","mean","min","max"])
print(summarize)
print("=======Multiple Statistics=======")
multiple=df.groupby('Department')['Performance
Score'].agg(["count","mean","min","max","sum","median","std"])
print(multiple)
print("=======Sort Values To Rank Employees=======")
```

```
df=df.sort_values(by='Performance Score',ascending=False)

df["Rank"]=range(1,len(df)+1)

print(df)

print("=======Loc And I Loc to Filter Records=======")

print(df.loc[df['Performance Score']>10])

print(df.loc[df['Department']=='Finance'])

print(df.iloc[df['Job Role']=='Data Analyst'])

print(df.iloc[3])

print(df.loc[3])

print(df.iloc[:3,:0])


print("=======Pivot Table Compare Salary And Performance=======")

pivot=df.pivot_table(index=['Performance Score'],values=['Monthly Salary'],aggfunc='sum')

print(pivot)


print("=======Merge To Combine Employee And Department Budget Datasets=======")

budget=pd.DataFrame({"Department":["IT","HR","Sales","Finance","Marketing"],"Department_Budget":[
25000,23000,45000,26000,55000]})

print(budget)

m=pd.merge(df,budget,on="Department",how="left").merge

print(m)


print("=======Apply Vectorized Operations To Classify Employees=======")

df['Employee ID']=df['Performance Score'].apply(lambda x:"Excellent" if x>=8 else ("Average" if x>=5
else "Needs Improvement"))

print(df)


print("==============================Business
Deliverables==============================")

print("=======Cleaned Dataset To Cleaned Employee Data.xlsx=======")

df.to_excel("cleaned_employee_data.xlsx",index=False)
```

```
print("=======Department Summaries To A Separated Excel Sheet=======")
summar1=df.groupby('Department')['Monthly Salary'].count().sort_values(ascending=False)
summar2=df.groupby('Department')['Monthly Salary'].mean().sort_values(ascending=False)
summar3=df.groupby('Department')['Monthly Salary'].sum().sort_values(ascending=False)
summar4=df.groupby('Department')['Performance Score'].count().sort_values(ascending=False)
summar5=df.groupby('Department')['Gender'].count().sort_values(ascending=False)
print(summar1,summar2,summar3,summar4,summar5)
with pd.ExcelWriter("Department_Summaries.xlsx") as writer:
  summar1.rename("Total_Employee").to_frame().to_excel(writer,sheet_name="Employee_count")
  summar2.rename("Average_Salary").to_frame().to_excel(writer,sheet_name="Average_Salary")
  summar3.rename("Total_Salary").to_frame().to_excel(writer,sheet_name="Total_Salary")

summar4.rename("Emp_Performance_Score_Count").to_frame().to_excel(writer,sheet_name="Emp_
Performance_Score_Count")

summar5.rename("Emp_Gender_Count").to_frame().to_excel(writer,sheet_name="Emp_Gender_Cou
nt")
  print("==================All Department Summaries Export Successfully==============")

print("=======Top 10 Performance In A Separate Sheet=======")
top_10=df.sort_values(by="Performance Score",ascending=False).head(10)
print(top_10)
print("==================Top 10 Performance Export Successfully====================")

print("======Fianl Report Containing At Least 10 Business insights=======")
print("======Business Insights Report=======")
totald=df['Department'].count()
print("Total Department:",totald)
emp_depart=df.groupby('Department')['Employee ID'].count().sort_values(ascending=False)
print("Employee Per Department:",emp_depart)
avg_monthly_sala=df['Monthly Salary'].mean()
print("Average Monthly Salary:",avg_monthly_sala)
```

```
highest_paid=df.groupby('Employee Name')['Monthly
Salary'].sum().sort_values(ascending=False).head(10)

print("Highest Paid Employee:",highest_paid)

lowest_paid=df.groupby('Employee Name')['Monthly Salary'].sum().nsmallest(10)

print("Lowest Paid Employee:",lowest_paid)

avg_performance=df.groupby('Employee ID')['Performance
Score'].mean().sort_values(ascending=False)

print("Average Performance Employee:",avg_performance)

top=df.groupby('Employee ID')['Performance Score'].sum().sort_values(ascending=False)

print("Top Performing Employees:",top)

dep_per=df.groupby('Department')['Performance Score'].mean().sort_values(ascending=False)

print("Department Average Performance:",dep_per)

salary_exp=df.groupby('Department')['Monthly Salary'].sum().sort_values(ascending=False)

print("Salary Expense:",salary_exp)

print("Department Budget:",budget)

depar_gen=df.groupby('Department')['Gender'].count().sort_values(ascending=False)

print("Department Wise Gender:",depar_gen)

job_sala=df.groupby('Job Role')['Monthly Salary'].sum().sort_values(ascending=False)

print("Job Role Monthly Total Salary:",job_sala)


print("==============================Business
Questions==============================")

print("=======Department Has The Highest Average Salary=======")

dep_sala=df.groupby('Department')['Monthly Salary'].mean().sort_values(ascending=False)

print(dep_sala)

print("=======Job Role Has The Highest Salary=======")

job_salary=df.groupby('Job Role')['Monthly Salary'].sum().sort_values(ascending=False)

print(job_salary)

print("=======Employee With More Experience Have Higher Salaries=======")

experi_hig=df.groupby('Experience Years')['Monthly Salary'].mean().sort_values(ascending=False)

print(experi_hig)

print("=======Which Department Has The Highest Attrition Rate=======")
```

```
high_attrition=df.groupby('Department')['Attrition'].count().sort_values(ascending=False)
print(high_attrition)
print("=======Employees Have a Performance Score Above 8 & Experience Below 3 Years=======")
result=df.loc[(df['Performance Score']>8)&(df['Experience Years']<3)]
print(result)
print("=======Percentage of Employees Earn More Than 50000 Per Month======")
high_salar=df.loc[(df['Monthly Salary']>50000)]
percentage=(len(high_salar)/len(df))*100
print("Employee Earn 50000:",len(high_salar))
print("Total Employee:",len(df))
print("Percentage Employee:",round(percentage,2),"%")
print("=======Department has The Highest Average Performance Score=======")
avg_per=df.groupby('Department')['Performance Score'].mean().sort_values(ascending=False)
print(avg_per)
print("=======Employee Completing More Projects Associated With higher Performance Score=======")
project=df.groupby('Project Completed')['Performance Score'].count().sort_values(ascending=False)
print(project)
print("=======Employees May Require Performance Improvement Plans=======")
prfor=df.loc[(df['Performance Score']<5)]
print(prfor)
print("=======HR Manager Based On The Findings=======")
print("Identify employee with low performance scores")
print("Recognize and reward high-performing employees")
print("Analyze the relationship between experience")
print("Review employees earning above 50000 per month")
print("============End Of Recomemendations============")

print("=======Python And Pandas Skills Covered=======")
```

```
print("==============================5 Project Submission Checklist==============================")

print("=======Original Excel Dataset=======")

df=pd.read_excel("employee_data_original.xlsx")

print(df)
```